

# WasmBox - Week 1 Blueprint

WASM Foundation + React/Monaco Developer Portal

Based on the supplied Project 3 specification: Week 1 establishes the Wasmtime-Python runtime, executes a precompiled benign WASM module from a Python host, and scaffolds the React/Monaco developer interface.

## 1. Week 1 Goal

The first milestone is to prove that the backend can load and execute a safe, precompiled WebAssembly module through Wasmtime, while the frontend establishes the browser-based plugin development experience. The full Python-to-WASM compiler is intentionally a Week 2 activity.

| STEP 1   | Wasmtime-Python  | Set up the Python host runtime            |
|----------|------------------|-------------------------------------------|
| STEP 2   | Precompiled WASM | Load a benign module such as hello.wasm   |
| STEP 3   | Execution        | Instantiate and run the exported function |
| STEP 4   | Result           | Return a simple result such as 42         |
| PARALLEL | React + Monaco   | Scaffold the developer portal UI          |

## 2. Week 1 Architecture Blueprint

```
DEVELOPER
↓
React Frontend + Monaco Editor
↓
Python Host Application
↓
Wasmtime-Python Runtime
↓
Precompiled hello.wasm
↓
WASM Function Execution
↓
Execution Result (example: 42)
```

## 3. Backend Components

| Component              | Purpose                                                                                         |
|------------------------|-------------------------------------------------------------------------------------------------|
| Wasmtime-Python        | Embeds the WebAssembly runtime into the Python host application.                                |
| Python Host            | Creates the engine/store, loads the module, instantiates it, and invokes the exported function. |
| hello.wat / hello.wasm | Small benign demonstration module used to verify the execution foundation.                      |
| Execution Result       | Captures the return/output of the test module and proves that the runtime path works.           |

## 4. Week 1 Project Structure

```
backend/
  app/
  main.py
```

```

■ ■■■ sandbox.py
■■■ wasm/
■■■ hello.wat

frontend/
■■■ src/
■ ■■■ App.jsx
■ ■■■ api.js
■ ■■■ main.jsx
■ ■■■ styles.css
■■■ package.json

```

## 5. Example Week 1 Host Code

A minimal host follows this lifecycle:

```

import wasmtime

engine = wasmtime.Engine()
module = wasmtime.Module(engine, wasm_bytes)
store = wasmtime.Store(engine)

# instantiate module and call exported function

```

## 6. React + Monaco Developer Portal

The frontend is scaffolded as a browser-based development environment. Monaco provides a VS Code-like editor where a developer can type plugin source. At this stage, the editor is the foundation for the Week 2 execution workflow rather than the complete compiler.

| Developer Portal Area   | Week 1 role                                                     |
|-------------------------|-----------------------------------------------------------------|
| Header / project status | Identifies WasmBox and sandbox context.                         |
| Tenant / plugin inputs  | Introduces multi-tenant/plugin concepts for the later pipeline. |
| Monaco Editor           | Provides the code editing surface.                              |
| Run action              | Reserved as the entry point for later backend execution.        |
| Output console          | Provides a place to display execution results.                  |

## 7. Week 1 Data Flow

Developer → React application → Monaco Editor → Python host → Wasmtime-Python → precompiled WASM → execution → result.

## 8. Week 1 Milestones and Evidence

| Milestone             | Expected evidence                                     |
|-----------------------|-------------------------------------------------------|
| Wasmtime-Python setup | Python environment imports Wasmtime successfully.     |
| Precompiled module    | hello.wat/hello.wasm exists and is loadable.          |
| WASM execution        | Host successfully invokes the exported test function. |
| Result                | A deterministic result such as 42 is observed.        |

| Milestone      | Expected evidence                                      |
|----------------|--------------------------------------------------------|
| React + Monaco | Frontend opens and displays the editor without errors. |

## 9. Week 1 → Week 2 Handoff

Week 1 proves the runtime foundation. Week 2 uses that foundation to accept raw plugin source, pass it through a compiler, produce executable WASM, run it through Wasmtime, and return stdout to the React interface.

Week 1: Precompiled WASM → Wasmtime → Result

Week 2: User Python → FastAPI → Compiler → WASM → Wasmtime → stdout → React

## 10. Mid-Review Readiness

After Week 2, the project moves into the security audit. The supplied brief calls for testing a malicious plugin against /etc/passwd access and an external socket connection, proving both are denied. Week 3 then adds strict resource limits and metrics, while Week 4 adds narrowly whitelisted host functions, persistence, and simulated webhooks.